

Kubeflow Pipelines - GitHub 問題摘要

來源：<https://codelabs.developers.google.com/codelabs/cloud-kubeflow-pipelines-gis?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將使用 GKE 設定 Cloud AI 平台管道安裝作業 (託管 KFP)、使用 Kubeflow Pipelines 建構及執行機器學習工作流程，以及在 AI 平台筆記本 (Jupyter) 中定義及執行管道。

目錄

1. [簡介](#)
2. [設定](#)
3. [從「管道」資訊主頁執行管道](#)
4. [從 AI Platform Notebook 執行管道](#)
5. [清除所用資源](#)
6. [附錄](#)

1. 簡介

	Kubeflow 是 Kubernetes 專用的機器學習工具包。這項專案旨在簡化在 Kubernetes 中部署機器學習 (ML) 工作流程的作業，並提高其可攜性與可擴充性。此專案目標是提供直觀的方式，協助使用者將領先業界的 ML 專屬開放原始碼系統部署至各個不同的基礎架構。
 Kubeflow	機器學習工作流程可能包含許多步驟，彼此之間存在依附元件，從資料準備和分析，到訓練、評估、部署等等。以臨時方式 (例如在一組筆記本或指令碼中) 撰寫及追蹤這些程序並不容易，稽核和重現性等事項也會日益成問題。Kubeflow Pipelines (KFP) 提供部署穩健且可重複執行的機器學習管道的方法，以及監控、稽核、版本追蹤和重現性等功能，有助於解決這些問題。Cloud AI Pipelines 可讓您輕鬆設定 KFP 安裝作業。

建構項目

在本程式碼研究室中，您將建構一個網路應用程式，使用 [Kubeflow Pipelines](#) 訓練及提供模型，以摘要說明 GitHub 問題。這個範例是以 [Kubeflow Examples 存放區](#) 中的範例為基礎。完成後，基礎架構會包含：

- 已安裝 Kubeflow Pipelines 的 [Google Kubernetes Engine \(GKE\)](#) 叢集 (透過 [Cloud AI Pipelines](#) 安裝)。
- 在 GPU 上訓練 [Tensor2Tensor](#) 模型的管道
- 提供訓練模型預測結果的服務容器
- 可解讀預測結果的 UI，為 GitHub 問題提供摘要
- 筆記本：使用 [Kubeflow Pipelines \(KFP\) SDK](#) 從頭建立 pipeline

課程內容

您建構的管道會根據 GitHub 問題資料訓練 [Tensor2Tensor](#) 模型，學習根據問題內容預測問題標題。接著，它會匯出訓練好的模型，並使用 [Tensorflow Serving](#) 部署匯出的模型。pipeline 的最後一個步驟會啟動網頁應用程式，與 TF-Serving 執行個體互動，以取得模型預測結果。

- 如何在 GKE 叢集上安裝 Kubeflow Pipelines
- 如何使用 Kubeflow Pipelines 建構及執行機器學習工作流程
- 如何從 [AI 平台筆記本](#) 定義及執行管道

軟硬體需求

- 瞭解 [Kubernetes](#) 的基本概念會有幫助，但並非必要
- 您擁有「擁有者」權限的有效 [GCP 專案](#)
- (選用) [GitHub](#) 帳戶

- 可以存取 [Google Cloud Shell](#)，這項工具位於 [Google Cloud Platform \(GCP\) Console](#)
-

步驟 2 · 約 10 分鐘

2. 設定

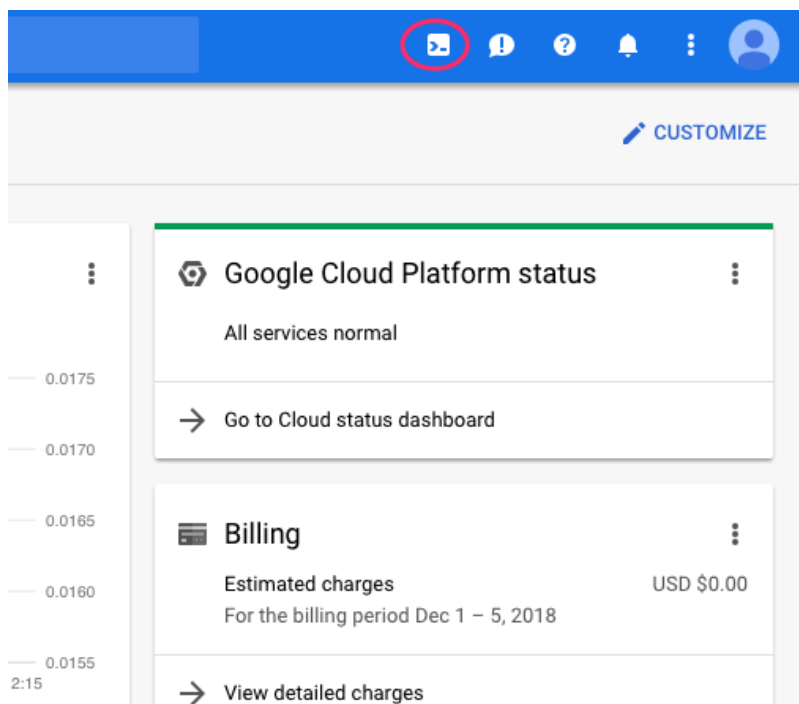
Cloud Shell

在瀏覽器中前往 GCP 主控台，並使用專案憑證登入：

視需要按一下「選取專案」，確保您使用的是程式碼研究室專案。

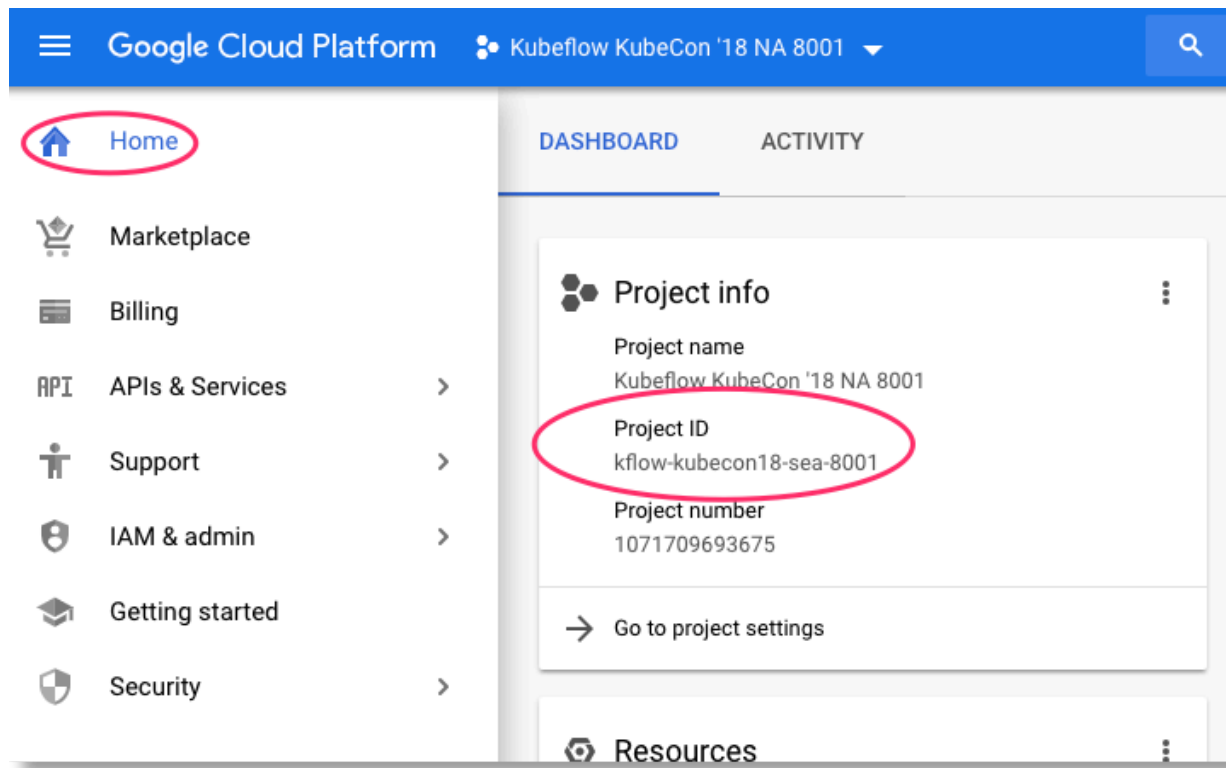


然後點選控制台右上角的「啟用 Cloud Shell」圖示，啟動 [Cloud Shell](#)。



啟動 Cloud Shell 時，系統會顯示目前使用的專案名稱。確認這項設定是否正確。

如要尋找專案 ID，請前往 GCP 主控台的「首頁」面板。如果畫面空白，請在提示中按一下「是」建立資訊主頁。



接著，視需要執行下列指令，將 `gcloud` 設定為使用正確的專案：

```
export PROJECT_ID=<your_project_id>
gcloud config set project ${PROJECT_ID}
```

建立儲存空間 bucket

注意：bucket 名稱在整個 GCP 中皆不得重複，而不只是在貴機構內。

建立 [Cloud Storage](#) bucket，用來儲存管道檔案。您必須使用全域不重複的 ID，因此定義包含專案 ID 的 bucket 名稱很方便。使用 `gsutil mb` (建立值區) 指令建立值區：

```
export PROJECT_ID=<your_project_id>
export BUCKET_NAME=kubeflow-${PROJECT_ID}
gsutil mb gs://${BUCKET_NAME}
```

或者，您也可以透過 [GCP 主控台](#) 建立 bucket。

選用**：建立 GitHub 權杖**

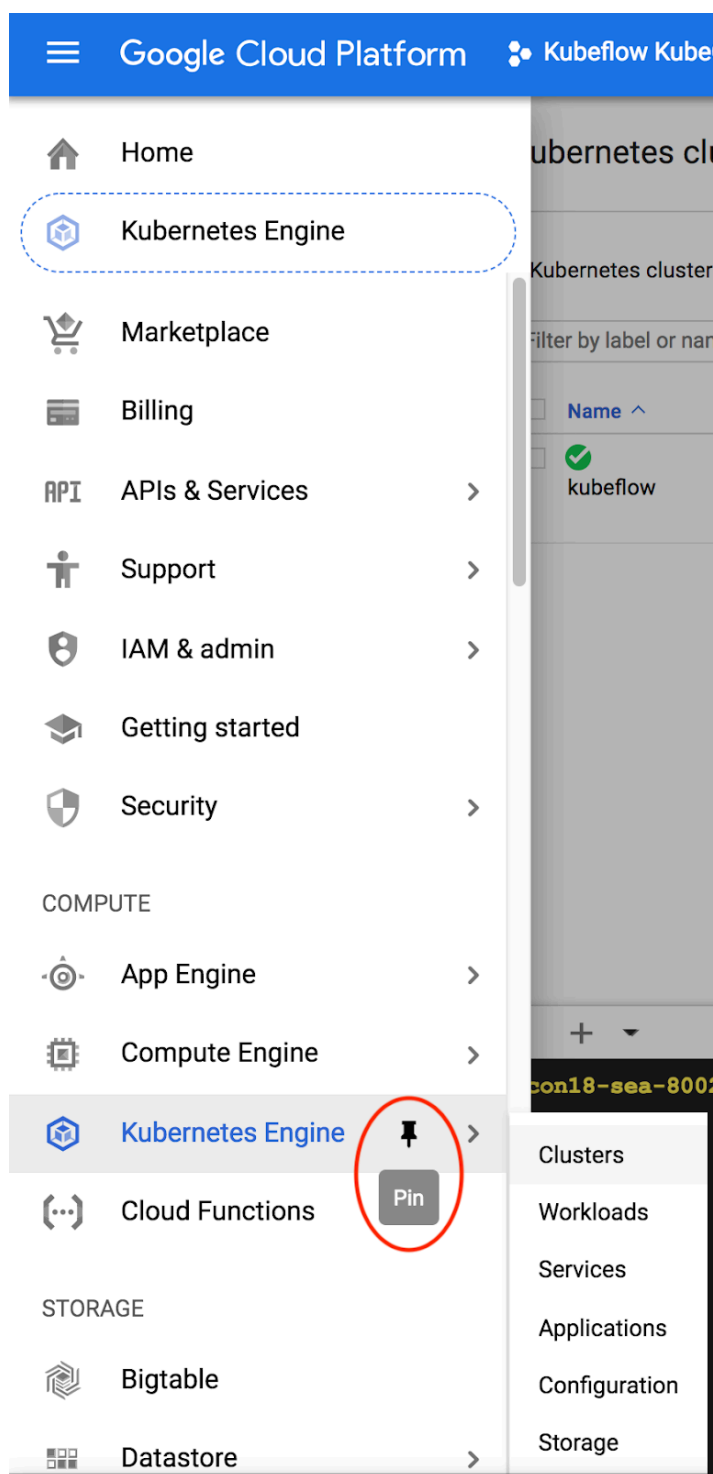
本程式碼研究室會呼叫 GitHub API，擷取公開資料。為避免受到速率限制，特別是在大量匿名要求傳送至 GitHub API 的活動中，請設定沒有權限的存取權權杖。這只是為了授權您個人，而非匿名使用者。

1. 前往 <https://github.com/settings/tokens>，產生沒有範圍的新權杖。
2. 請將救援碼儲存在安全的地方。如果遺失，請刪除並建立新金鑰。

如果略過這個步驟，實驗室仍可運作，只是在產生輸入資料來測試模型時，選項會比較有限。

選用：釘選實用資訊主頁

在 GCP 主控台中，釘選「Kubernetes Engine」和「Storage」資訊主頁，方便存取。



建立 AI 平台管道 (託管 Kubeflow 管道) 安裝項目

請按照[這個網頁](#)的「事前準備」和「設定執行個體」一節中的說明，設定已安裝 KFP 的 GKE 執行個體。請務必勾選「允許存取下列 Cloud API」方塊，如說明文件所示。(否則範例管道將無法順利執行)。將安裝命名空間保留為 `default`。

您需要選取支援 Nvidia k80 的區域。您可以將 `us-central1-a` 或 `us-central1-c` 設為預設值。

安裝完成後，請記下 [AI Pipelines 資訊主頁](#)中列出的 GKE 叢集名稱和區域，並為這些值設定環境變數，方便後續使用。

Filter ?							
☐	Status	Name ↑		Zone	Version	Cluster	Namespace
☐	✓	kubeflow-pipelines-1	OPEN PIPELINES DASHBOARD	us-central1-c	1.0.0	cluster-1	default SETTINGS

```
export ZONE=<your zone>
export CLUSTER_NAME=<your cluster name>
```

設定 kubectl 以使用新 GKE 叢集的憑證

建立 GKE 叢集後，請在 Cloud Shell 中執行下列指令，設定 `kubectl` 使用新叢集的憑證：

```
gcloud container clusters get-credentials ${CLUSTER_NAME} \
  --project ${PROJECT_ID} \
  --zone ${ZONE}
```

或者，按一下 AI Pipelines 資訊主頁中的叢集名稱，前往該叢集的 GKE 頁面，然後按一下頁面頂端的「連線」。在彈出式視窗中，將指令貼入 Cloud Shell。

這會設定 `kubectl` 環境，方便您與叢集互動。如要驗證設定，請執行下列指令：

```
kubectl get nodes -o wide
```

您應該會看到節點列出，狀態為「Ready」，以及節點年齡、版本、外部 IP 位址、OS 映像檔、核心版本和容器執行階段等其他資訊。

設定叢集，在啟用 GPU 的節點集區中安裝 Nvidia 驅動程式

接著，我們將會將 `daemonset` 套用至叢集，在所有啟用 GPU 的叢集節點上安裝 Nvidia 驅動程式：

```
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/container-engine-accelerators/master/nvidia-driver-installer/cos/daemonset-preloaded.yaml
```

接著執行下列指令，授予 KFP 元件建立新 Kubernetes 資源的權限：

```
kubectl create clusterrolebinding sa-admin --clusterrole=cluster-admin --
serviceaccount=kubeflow:pipeline-runner
```

建立 GPU 節點集區

接著，我們要設定大小為 1 的 GPU 節點集區：

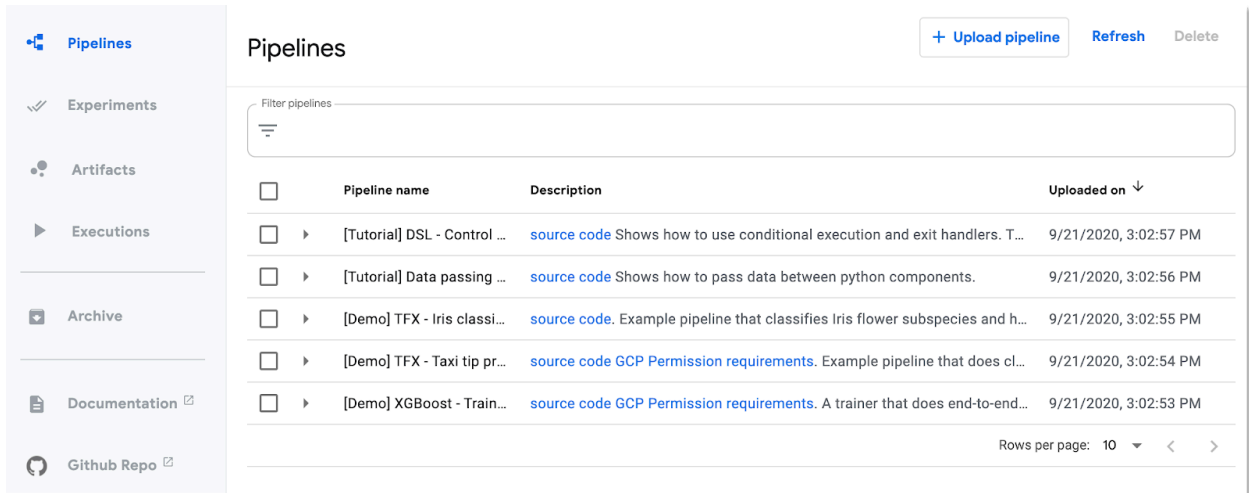

```
gcloud container node-pools create gpu-pool \  
  --cluster=${CLUSTER_NAME} \  
  --zone ${ZONE} \  
  --num-nodes=1 \  
  --machine-type n1-highmem-8 \  
  --scopes cloud-platform --verbosity error \  
  --accelerator=type=nvidia-tesla-k80,count=1
```

注意：這項作業可能要花 5 分鐘的時間。如果在建立節點集區時看到 GPU 配額錯誤，可以[申請更多配額](#)。

3. 從「管道」資訊主頁執行管道

開啟管道資訊主頁

如果尚未前往 Cloud 控制台，請前往 [Pipelines 面板](#)。然後點選安裝項目的「OPEN PIPELINES DASHBOARD」(開啟管線資訊主頁)，並點選左選單列中的「Pipelines」(管線)。如果發生載入錯誤，請重新整理分頁。您應該會看到類似下方的頁面：



管道說明

您將執行的管道包含幾個步驟 (詳情請參閱本程式碼研究室的附錄)：

1. 現有模型檢查點會複製到值區。
 2. 使用預先處理的資料訓練 [Tensor2Tensor](#) 模型。
- 訓練作業會從第一個步驟中複製的現有模型檢查點開始，然後再訓練幾百個步驟。(在本程式碼研究室中完整訓練模型會花費太多時間)。
 - 訓練完成後，管道步驟會匯出模型，並以適合 [TensorFlow Serving](#) 的形式提供模型。
3. 並使用該模型部署 TensorFlow Serving 執行個體。
 4. 啟動網頁應用程式，與提供的模型互動，以擷取預測結果。

下載及編譯管道

在本節中，我們將瞭解如何編譯管道定義。首先，我們需要安裝 KFP SDK。在 Cloud Shell 中執行下列指令：

```
pip3 install -U kfp
```

若要下載管道定義檔案，請從 Cloud Shell 執行下列指令：

```
curl -O https://raw.githubusercontent.com/amygdala/kubeflow-examples/ghsumm/github_issue_summarization/pipelines/example_pipelines/gh_summ_hosted_kfp.py
```

接著，執行管道定義檔案，如下所示：

```
python3 gh_summ_hosted_kfp.py
```

結果會顯示檔案 `gh_summ_hosted_kfp.py.tar.gz`。

注意：如果發生錯誤，請確認您已安裝 Pipelines SDK，並使用 Python 3。

上傳已編譯的管道

在 Kubeflow Pipelines 網頁介面中，按一下「Upload pipeline」，然後選取「Import by URL」。複製下列網址，然後貼上。這個網址指向您剛編譯的相同管道。(從 Cloud Shell 上傳檔案需要幾個額外步驟，因此我們採取捷徑)。

https://storage.googleapis.com/aju-dev-demos-codelabs/KF/compiled_pipelines/gh_summ_hosted_kfp.py.tar.gz

為管道命名 (例如 `gh_summ`)。

Pipeline Versions

← Upload Pipeline or Pipeline Version

☒ Create a new pipeline ☐ Create a new pipeline version under an existing pipeline

Upload pipeline with the specified package.

Pipeline Name *

gh_summ

Pipeline Description *

URL must be publicly accessible.

For expected file format, refer to [Compile Pipeline Documentation](#).

☐ Upload a file File * Choose file

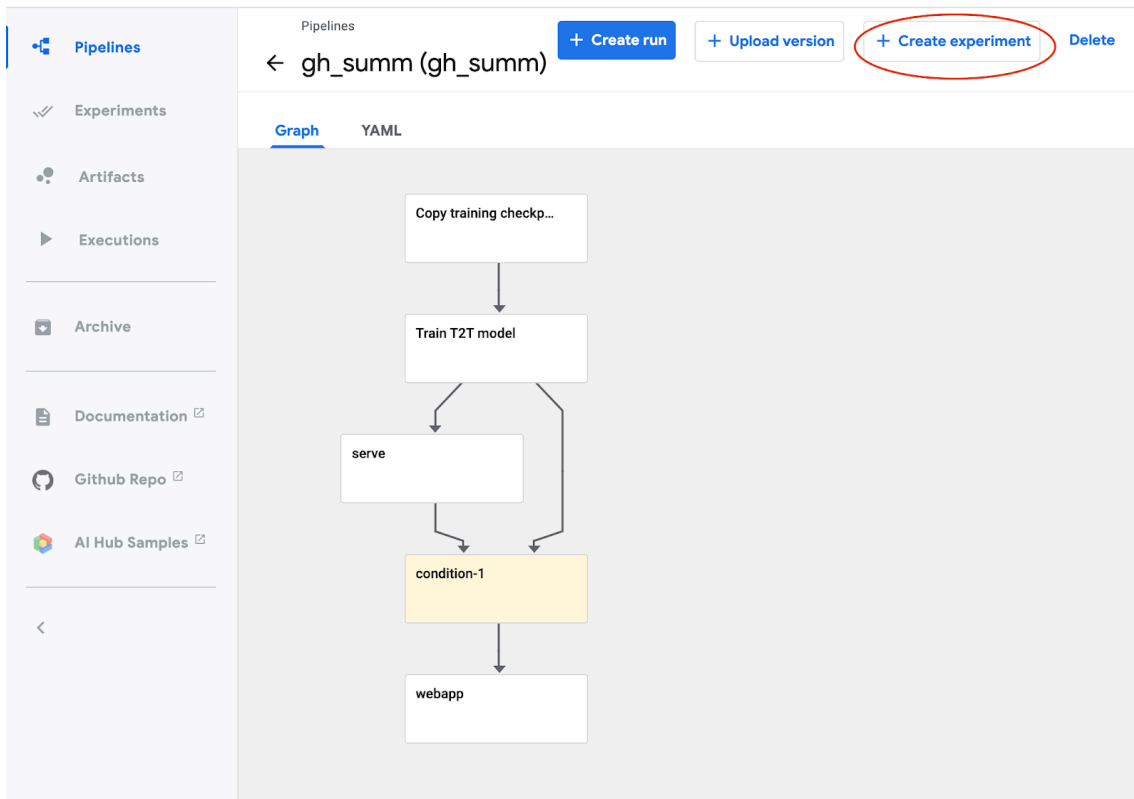
☒ Import by url Package Url

https://storage.googleapis.com/aju-dev-demos-codelabs/KF/compiled_pipelines/gh_summ_hosted_kfp.py.tar.gz

Code Source (optional)

執行管道

在清單中按一下已上傳的管道 (這會顯示管道的靜態圖)，然後按一下「建立實驗」，使用管道建立新的「實驗」。實驗可將語意相關的執行作業分組。



為實驗命名 (例如與管道相同的名稱 `gh_summ`)，然後按一下「Next」(下一步) 建立實驗。

The screenshot shows the 'New experiment' form in the Kubeflow Experiments section. The form has a title 'New experiment' and a section 'Experiment details'. It includes a text input for 'Experiment name *' with the value 'gh_summ', and a text input for 'Description (optional)'. At the bottom are 'Next' and 'Cancel' buttons.

系統會顯示頁面，供您輸入 Run 的參數並啟動。

您可能需要在 Cloud Shell 中執行下列指令，協助填入參數。

```
gcloud config get-value project
echo "gs://${BUCKET_NAME}/codeclab"
```

系統會自動填入執行名稱，但您也可以視需要變更名稱。

然後填寫三個參數欄位：

- `project`
- (選填) `github-token`

- `working-dir`

在 `working-dir` 中，輸入您建立的 GCS bucket 下方的路徑。請加入「`gs://`」前置字元。在 `github-token` 欄位中，輸入您先前選擇性產生的權杖，或將預留位置字串保留原樣 (如果您未產生權杖)。

Run parameters

Specify parameters required by the pipeline

train_steps	2019300
project	YOUR_PROJECT_HERE
github_token	YOUR_GITHUB_TOKEN_HERE
working_dir	gs://YOUR_GCS_DIR_HERE
checkpoint_dir	gs://aju-dev-demos-codelabs/kubecon/model_output_tbase.bak2019000/
deploy_webapp	true
data_dir	gs://aju-dev-demos-codelabs/kubecon/t2t_data_gh_all/

StartCancel

填寫完欄位後，按一下「開始」，然後點選列出的執行作業來查看詳細資料。在特定管道步驟執行期間，您可以點選該步驟來取得更多資訊，包括查看 [Pod](#) 記錄。(您也可以透過管道步驟的 [Cloud Logging \(Stackdriver\)](#) 記錄連結查看記錄，即使叢集節點已終止也沒問題)。

Pipelines

Experiments

Artifacts

Executions

Archive

Documentation

Github Repo

AI Hub Samples

Experiments > e2

Run of gh_summ_hosted_kfp (e66aa)

GraphRun outputConfig

Copy training chec...

✓

Train T2T model

🔄

...

github-issue-summarization-c4q6z-2891853010

Input/OutputVisualizationsML MetadataVolumesLogsPodEvents

Logs can also be viewed in Stackdriver Kubernetes Monitoring.

74 ::::MLPv0.5.0 transformer 1600899995.829186108 (/usr/local/lib/python2.7/dist-packages/tensor2t

79 ::::MLPv0.5.0 transformer 1600899996.240576982 (/usr/local/lib/python2.7/dist-packages/tensor2t

80 ::::MLPv0.5.0 transformer 1600899996.241978884 (/usr/local/lib/python2.7/dist-packages/tensor2t

81 ::::MLPv0.5.0 transformer 1600899996.243154049 (/usr/local/lib/python2.7/dist-packages/tensor2t

82 ::::MLPv0.5.0 transformer 1600899996.652068988 (/usr/local/lib/python2.7/dist-packages/tensor2t

83 ::::MLPv0.5.0 transformer 1600899996.653433084 (/usr/local/lib/python2.7/dist-packages/tensor2t

84 ::::MLPv0.5.0 transformer 1600899996.654628038 (/usr/local/lib/python2.7/dist-packages/tensor2t

85 ::::MLPv0.5.0 transformer 1600899997.250725985 (/usr/local/lib/python2.7/dist-packages/tensor2t

86 ::::MLPv0.5.0 transformer 1600899997.252022028 (/usr/local/lib/python2.7/dist-packages/tensor2t

87 ::::MLPv0.5.0 transformer 1600899997.253187895 (/usr/local/lib/python2.7/dist-packages/tensor2t

88 ::::MLPv0.5.0 transformer 1600899997.672970057 (/usr/local/lib/python2.7/dist-packages/tensor2t

89 ::::MLPv0.5.0 transformer 1600899997.674216032 (/usr/local/lib/python2.7/dist-packages/tensor2t

90 ::::MLPv0.5.0 transformer 1600899997.675436974 (/usr/local/lib/python2.7/dist-packages/tensor2t

91 ::::MLPv0.5.0 transformer 1600899998.004769964 (/usr/local/lib/python2.7/dist-packages/tensor2t

92 ::::MLPv0.5.0 transformer 1600899998.006061954 (/usr/local/lib/python2.7/dist-packages/tensor2t

93 ::::MLPv0.5.0 transformer 1600899998.007261915 (/usr/local/lib/python2.7/dist-packages/tensor2t

94 ::::MLPv0.5.0 transformer 1600899998.212174892 (/usr/local/lib/python2.7/dist-packages/tensor2t

95 ::::MLPv0.5.0 transformer 1600899998.333297968 (/usr/local/lib/python2.7/dist-packages/tensor2t

96 ::::MLPv0.5.0 transformer 1600899998.343330994 (/usr/local/lib/python2.7/dist-packages/tensor2t

97 ::::MLPv0.5.0 transformer 1600899998.344532967 (/usr/local/lib/python2.7/dist-packages/tensor2t

98 ::::MLPv0.5.0 transformer 1600899998.345788956 (/usr/local/lib/python2.7/dist-packages/tensor2t

99 ::::MLPv0.5.0 transformer 1600899999.087954044 (/usr/local/lib/python2.7/dist-packages/tensor2t

100 ::::MLPv0.5.0 transformer 1600899999.089333057 (/usr/local/lib/python2.7/dist-packages/tensor2t

101 ::::MLPv0.5.0 transformer 1600899999.090585947 (/usr/local/lib/python2.7/dist-packages/tensor2t

102 ::::MLPv0.5.0 transformer 1600899999.729526043 (/usr/local/lib/python2.7/dist-packages/tensor2t

103 ::::MLPv0.5.0 transformer 1600899999.730871916 (/usr/local/lib/python2.7/dist-packages/tensor2t

104 ::::MLPv0.5.0 transformer 1600899999.732117891 (/usr/local/lib/python2.7/dist-packages/tensor2t

105 ::::MLPv0.5.0 transformer 1600900000.588685989 (/usr/local/lib/python2.7/dist-packages/tensor2t

106 ::::MLPv0.5.0 transformer 1600900000.590126991 (/usr/local/lib/python2.7/dist-packages/tensor2t

107 ::::MLPv0.5.0 transformer 1600900000.591386000 (/usr/local/lib/python2.7/dist-packages/tensor2t

108 ::::MLPv0.5.0 transformer 1600900001.232150078 (/usr/local/lib/python2.7/dist-packages/tensor2t

109 ::::MLPv0.5.0 transformer 1600900001.233475924 (/usr/local/lib/python2.7/dist-packages/tensor2t

110 ::::MLPv0.5.0 transformer 1600900001.234642029 (/usr/local/lib/python2.7/dist-packages/tensor2t

111 ::::MLPv0.5.0 transformer 1600900001.875562973 (/usr/local/lib/python2.7/dist-packages/tensor2t

112 ::::MLPv0.5.0 transformer 1600900001.877260923 (/usr/local/lib/python2.7/dist-packages/tensor2t

113 ::::MLPv0.5.0 transformer 1600900001.878551006 (/usr/local/lib/python2.7/dist-packages/tensor2t

114 ::::MLPv0.5.0 transformer 1600900002.778774977 (/usr/local/lib/python2.7/dist-packages/tensor2t

115 ::::MLPv0.5.0 transformer 1600900002.780188004 (/usr/local/lib/python2.7/dist-packages/tensor2t

116 ::::MLPv0.5.0 transformer 1600900002.781478882 (/usr/local/lib/python2.7/dist-packages/tensor2t

117 ::::MLPv0.5.0 transformer 1600900002.883114100 (/usr/local/lib/python2.7/dist-packages/tensor2t

118 INFO:tensorflow:Transforming body output with symbol_model.tty_34299_512_top

119 ::::MLPv0.5.0 transformer 1600900003.055654049 (/usr/local/lib/python2.7/dist-packages/tensor2t

120 ::::MLPv0.5.0 transformer 1600900003.056619883 (/usr/local/lib/python2.7/dist-packages/tensor2t

121 INFO:tensorflow:Base learning rate: 2.000000

122 INFO:tensorflow:Trainable Variables Total size: 61681152

123 INFO:tensorflow:Non-trainable variables Total size: 5

124 INFO:tensorflow:Using optimizer Adam

注意：這個管道大約需要 15 分鐘才能完成。

查看管道定義

在管道執行期間，您可能想[進一步瞭解](#)管道的組合方式和執行作業。程式碼研究室的「附錄」一節提供更多詳細資料。

在 TensorBoard 中查看模型訓練資訊

訓練步驟完成後，選取「Visualizations」(視覺化) 分頁標籤，然後按一下藍色的「Start TensorBoard」(啟動 TensorBoard) 按鈕，準備就緒後，按一下「Open Tensorboard」(開啟 TensorBoard)。

Experiments > Default

← ✓ Run of gh_summ_hosted_kfp3 (b6087)

Graph Run output Config

```
graph TD; A[Copy training chec... ✓] --> B[Train T2T model ✓]; B --> C[serve ✓]; B --> D[webapp ✓]; C --> D;
```

✕

Input/Output **Visualizations** ML Metadata

Tensorboard

TF Version

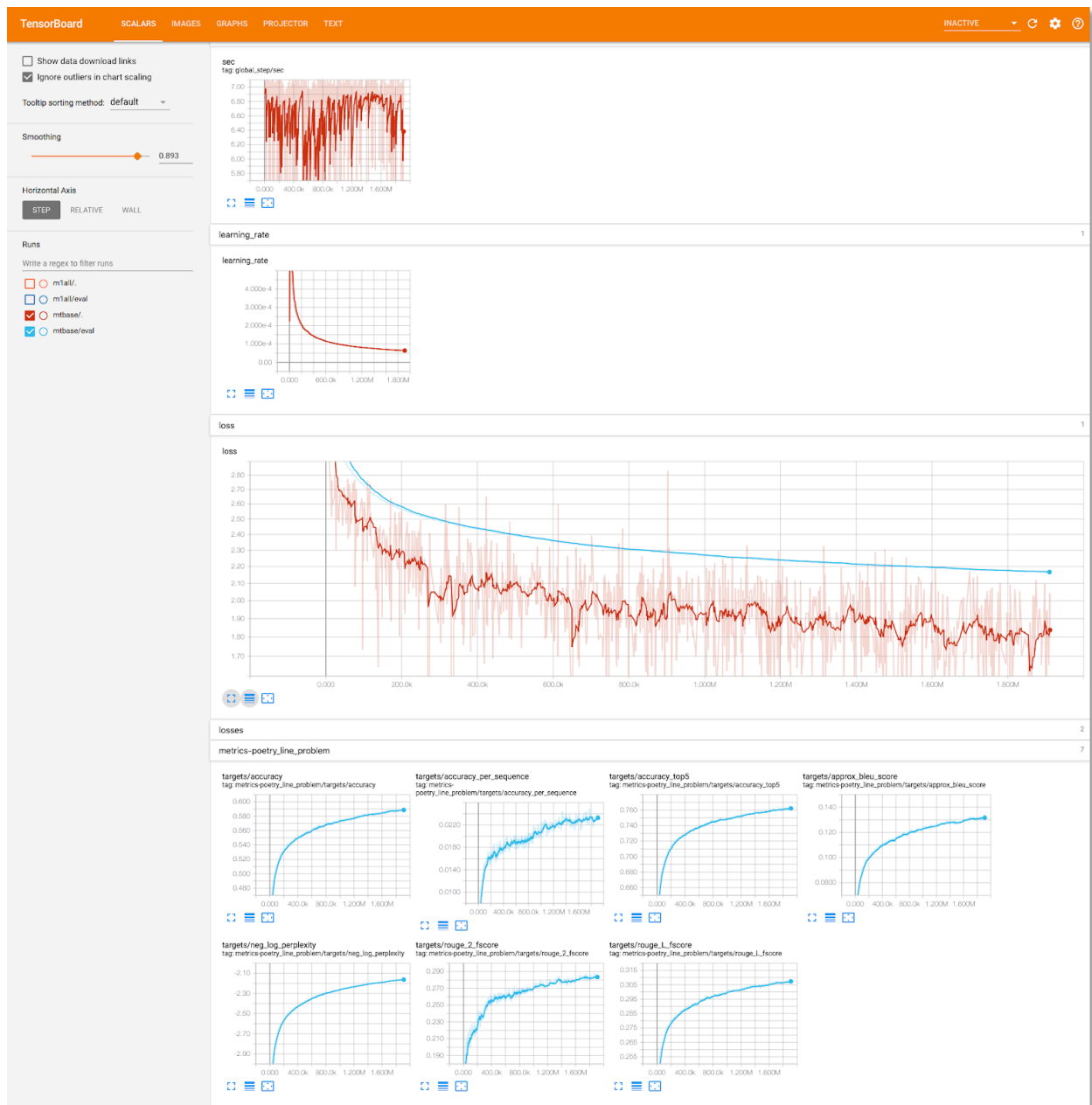
TensorFlow 2.0.0 ▾

Start Tensorboard

Visualization Creator

[create visualizations manually](#)

注意：如果 TensorBoard 分頁在幾分鐘後仍無法載入，請嘗試重新載入頁面。



探索構件和執行作業資訊主頁

Kubeflow Pipelines 會在管道執行時，自動記錄管道步驟的中繼資料。系統會記錄 **Artifact** 和 **Execution** 資訊。按一下資訊主頁左側導覽列中的這些項目，即可進一步探索。

Pipeline/Workspace	Name	ID	Type	URI	Created at
github-issue-summarization-gd...	copy_output_p...	9	GCSPath	minio://mlpipeline/artifacts/git...	9/21/2020, 4:2...
	main-logs	10	NoType	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:2...
	mlpipeline-ui-m...	11	NoType	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:3...
	launch_server	12	String	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:3...
	train_output_p...	13	GCSPath	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:3...
	main-logs	14	NoType	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:3...
	main-logs	15	NoType	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:3...
	main-logs	16	NoType	minio://mlpipeline/artifacts/gith...	9/21/2020, 4:3...

如果是構件，您可以查看總覽面板和歷程探索器面板。

Artifacts

← launch_server

Overview Lineage Explorer

Type: String

URI
minio://mlpipeline/artifacts/github-issue-summarization-gdgks/github-issue-summarization-gdgks-598729998/train-t2t-model-launch_server.tgz

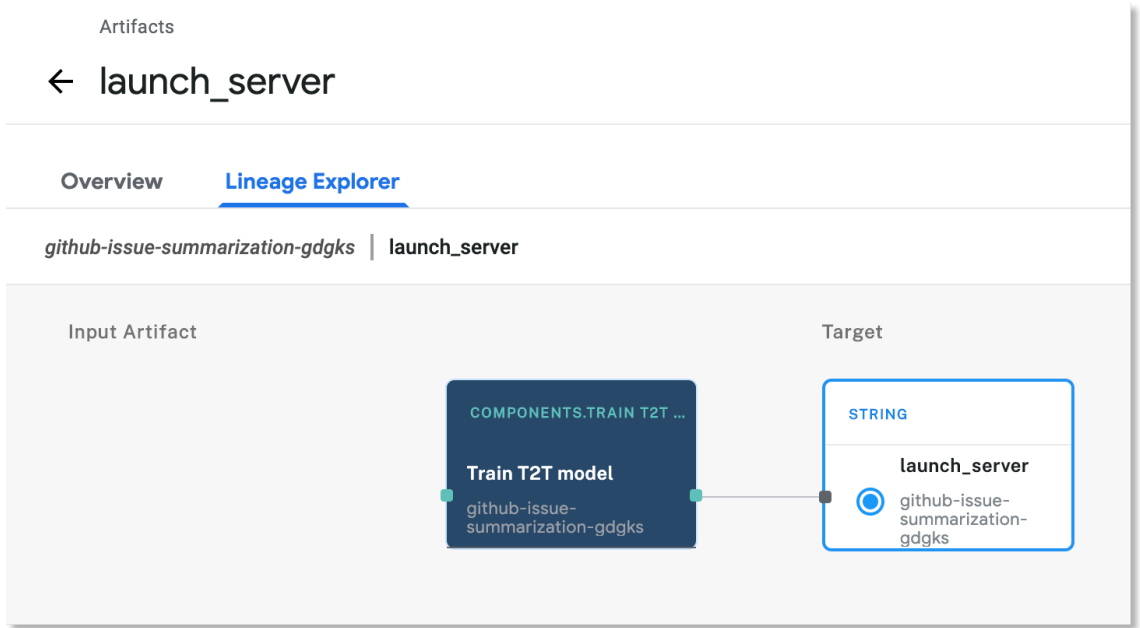
Properties

Custom Properties

argo_artifact

```
{
  "name": "train-t2t-model-launch_server",
  "path": "/tmp/output",
  "s3": {
    "accessKeySecret": {
      "key": "accesskey",
      "name": "mlpipeline-minio-artifact"
    },
    "bucket": "mlpipeline",
    "endpoint": "minio-service.kubeflow:9000",
    "insecure": true,
    "key": "artifacts/github-issue-summarization-gdgks/github-issue-summarization-gdgks-598729998/train-t2t-model-launch_server.tgz",
    "secretKeySecret": {
      "key": "secretkey",
      "name": "mlpipeline-minio-artifact"
    }
  }
}
```

name	pipeline_name	run_id
launch_server	github-issue-summarization-gdgks	github-issue-summarization-gdgks



啟動管道建立的網頁應用程式，並進行一些預測

pipeline 的最後一個步驟會部署網頁應用程式，提供查詢訓練模型的 UI (透過 [TF Serving](#) 提供)，以進行預測。

管道完成後，請轉送通訊埠至其 [服務](#)，連線至 Web 應用程式 (我們轉送通訊埠是因為在本程式碼研究室中，Web 應用程式服務未設定外部端點)。

在 Cloud Shell 中執行下列指令，找出服務名稱：

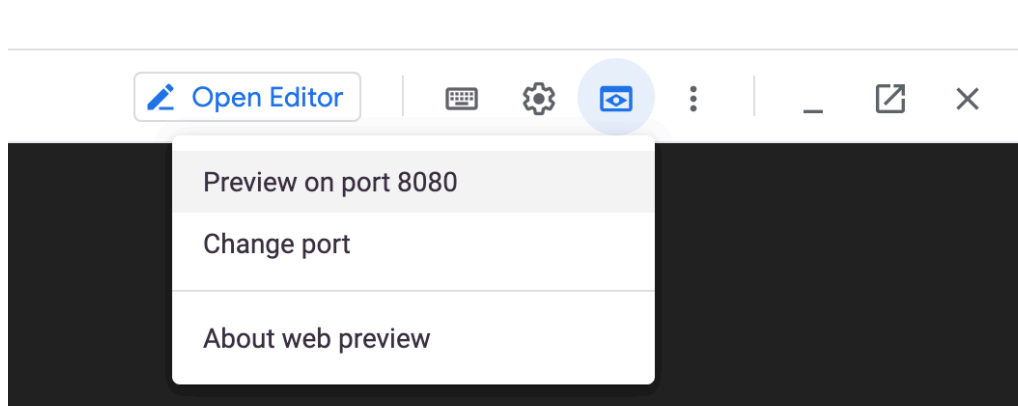
```
kubectl get services
```

在清單中尋找類似 `ghsumm-*-webappsvc` 的服務名稱。

接著，在 Cloud Shell 中，將通訊埠轉送至該服務，如下所示變更下列指令，使用您的 **webappsvc** 名稱：

```
kubectl port-forward svc/ghsumm-xxxxx-webappsvc 8080:80
```

通訊埠轉送作業執行完畢後，請點選 Cloud Shell 窗格上方的「預覽」圖示，然後在下拉式選單中點選「透過以下通訊埠預覽：8080」。



新分頁中應會顯示類似下方的頁面：

Github Issue Summarization

This app takes as input a Github issue body and predicts a title for it. Behind the scenes it uses a [Tensor2Tensor](#) TensorFlow model, served via [TF-Serving](#).

(Thanks to [Hamel Husain](#) for the original concept and source data.)

Enter the body of a github issue or the url of a github issue and click on Submit. The model then tries to generate a title or summary of the issue.

Enter Github Issue Body

Populate Random Issue

OR Enter Github Issue URL

<https://github.com/kubeflow/kubeflow/issues/232>

Generate Title

This demo is run using [Kubeflow](#) - a machine learning toolkit for Kubernetes. Kubeflow is dedicated to making deployment of machine learning on Kubernetes simple, portable and scalable.

按一下「Populate Random Issue」(填入隨機問題) 按鈕，即可擷取一段文字。按一下「Generate Title」(產生標題)，呼叫訓練好的模型並顯示預測結果。

Enter Github Issue Body

Populate Random Issue

hey there, i've installed this plugin and i was trying to use. however, i've followed your instruction but didn't work. i selected the line, clicked on right button and i could see the option send to terminal but the submenu is empty. i am using macos with the newest version updated. regards, fabio

OR Enter Github Issue URL

<https://github.com/kubeflow/kubeflow/issues/232>

Generate Title

Machine Generated Title

"not working on macos sierra"

注意：顯示摘要可能需要幾秒鐘，尤其是第一次提出要求時。在本研討會中，我們不會為 TensorFlow Serving 執行個體使用 GPU。

如果管道參數包含有效的 GitHub 權杖，您也可以嘗試在第二個欄位中輸入 GitHub 網址，然後按一下「產生標題」。如果沒有設定有效的 GitHub 權杖，請只使用「Populate Random Issue」欄位。

4. 從 AI Platform Notebook 執行管道

您也可以使用 KFP SDK，從 Jupyter 筆記本以互動方式定義及執行 Kubeflow Pipelines。 [AI Platform Notebooks](#) (本程式碼研究室將使用這項服務) 可讓您輕鬆完成這項作業。

建立筆記本執行個體

我們將使用 Cloud Shell 的 API 建立筆記本執行個體。(或者，您也可以[透過 Cloud Console](#) 建立筆記本。詳情請參閱[說明文件](#)。

在 Cloud Shell 中設定下列環境變數：

```
export INSTANCE_NAME="kfp-ghsumm"
export VM_IMAGE_PROJECT="deeplearning-platform-release"
export VM_IMAGE_FAMILY="tf2-2-3-cpu"
export MACHINE_TYPE="n1-standard-4"
export LOCATION="us-central1-c"
```

接著，在 Cloud Shell 中執行指令，建立筆記本執行個體：

```
gcloud beta notebooks instances create $INSTANCE_NAME \
  --vm-image-project=$VM_IMAGE_PROJECT \
  --vm-image-family=$VM_IMAGE_FAMILY \
  --machine-type=$MACHINE_TYPE --location=$LOCATION
```

首次執行這項指令時，系統可能會要求您為專案啟用 `notebooks` API。如果同意，請回覆「y」。

幾分鐘後，筆記本伺服器就會開始運作。您可以在 [Cloud Console](#) 中查看 [Notebook 執行個體](#)。

☐	☒	kfp-ghsumm	OPEN JUPYTERLAB	us-central1-c	4 vCPUs, 15 GB RAM
--------------------------	-------------------------------------	------------	---------------------------------	---------------	--------------------

上傳程式碼研究室筆記本

建立筆記本執行個體後，按一下[這個連結](#)，上傳程式碼研究室的 Jupyter 筆記本。選取要使用的筆記本執行個體。系統會自動開啟筆記本。

執行筆記本

按照筆記本中的指示完成實驗室的其餘部分。請注意，在筆記本的「設定」部分，您需要填入自己的值，才能執行筆記本的其餘部分。

(如果您使用自己的專案，請記得返回並完成本實驗室的「清除」部分)。

步驟 5 · 約 5 分鐘

5. 清除所用資源

如果您使用臨時 Codelab 帳戶，則不需要執行這項操作，但如果您使用自己的專案，可能需要移除 Pipelines 安裝項目和 Notebook。

關閉 Pipelines GKE 叢集

您可以從 [Cloud 控制台](#) 刪除 Pipelines 叢集。(如要重複使用 GKE 叢集，可以選擇只刪除 Pipelines 安裝項目)。

刪除 AI Notebook 執行個體

如果您已執行程式碼研究室的「筆記本」部分，可以從 [Cloud Console](#) 刪除或[停止](#)筆記本執行個體。

選用：移除 GitHub 權杖

前往 <https://github.com/settings/tokens>，然後移除產生的權杖。

6. 附錄

程式碼概覽

定義管道

本程式碼研究室使用的管道定義[請見這裡](#)。

讓我們看看如何定義轉換路徑，以及如何定義轉換路徑的元件 (步驟)。我們將介紹一些重點，詳情請參閱[說明文件](#)。

注意：雖然本範例未說明，但您可能也會對 KFP [輕量型 Python 元件](#)感興趣。輕量型 Python 元件不需要為每次程式碼變更建構新的容器映像檔，有助於快速疊代和原型設計。

Kubeflow Pipeline 步驟是以容器為基礎。建構管道時，您可以使用[預先建構的元件](#) (內含已建構的容器映像檔)，也可以自行建構元件。在本程式碼研究室中，我們已建構自己的應用程式。

其中四個管道步驟是從[可重複使用的元件](#)定義，並透過[元件定義檔案](#)存取。在第一個程式碼片段中，我們會透過網址存取這些元件定義檔案，並使用這些定義建立「ops」，用於建立管道步驟。

```
import kfp.dsl as dsl
import kfp.gcp as gcp
import kfp.components as comp

...

copydata_op = comp.load_component_from_url(

'https://raw.githubusercontent.com/kubeflow/examples/master/github_issue_summarization/pipelines/components/t2t/datacopy_component.yaml'
)

train_op = comp.load_component_from_url(

'https://raw.githubusercontent.com/kubeflow/examples/master/github_issue_summarization/pipelines/components/t2t/train_component.yaml'
)
```

以下是其中一個元件定義 (訓練作業)，格式為 YAML。您會看到已定義的輸入、輸出、容器映像檔和容器進入點引數。

```

name: Train T2T model
description: |
  A Kubeflow Pipeline component to train a Tensor2Tensor
  model
metadata:
  labels:
    add-pod-env: 'true'
inputs:
  - name: train_steps
    description: '...'
    type: Integer
    default: 2019300
  - name: data_dir
    description: '...'
    type: GCSPath
  - name: model_dir
    description: '...'
    type: GCSPath
  - name: action
    description: '...'
    type: String
  - name: deploy_webapp
    description: '...'
    type: String
outputs:
  - name: launch_server
    description: '...'
    type: String
  - name: train_output_path
    description: '...'
    type: GCSPath
  - name: MLPipeline UI metadata
    type: UI metadata
implementation:
  container:
    image: gcr.io/google-samples/ml-pipeline-t2ttrain:v3ap
    args: [
      --data-dir, {inputValue: data_dir},
      --action, {inputValue: action},
      --model-dir, {inputValue: model_dir},
      --train-steps, {inputValue: train_steps},
      --deploy-webapp, {inputValue: deploy_webapp},
      --train-output-path, {outputPath: train_output_path}
    ]
  env:
    KFP_POD_NAME: "{{pod.name}}"
  fileOutputs:
    launch_server: /tmp/output
    MLPipeline UI metadata: /mlpipeline-ui-metadata.json

```

您也可以透過 `dsl.ContainerOp` 建構函式定義管道步驟，如下所示。

以下是管道定義的大部分內容。我們將定義管道輸入內容 (及其預設值)。接著定義管道步驟。我們大多使用上述定義的「ops」，但也會透過 `ContainerOp` 定義「serve」步驟，直接指定容器映像檔和進入點引數。

您可以看到 `train`、`log_model` 和 `serve` 步驟會將前幾個步驟的輸出內容做為輸入內容。如要進一步瞭解如何指定這項設定，請參閱[這篇文章](#)。

```
@dsl.pipeline(
    name='Github issue summarization',
    description='Demonstrate Tensor2Tensor-based training and TF-Serving'
)
def gh_summ( #pylint: disable=unused-argument
    train_steps: 'Integer' = 2019300,
    project: str = 'YOUR_PROJECT_HERE',
    github_token: str = 'YOUR_GITHUB_TOKEN_HERE',
    working_dir: 'GCSPath' = 'gs://YOUR_GCS_DIR_HERE',
    checkpoint_dir: 'GCSPath' = 'gs://aju-dev-demos-
codelabs/kubecon/model_output_tbase.bak2019000/',
    deploy_webapp: str = 'true',
    data_dir: 'GCSPath' = 'gs://aju-dev-demos-codelabs/kubecon/t2t_data_gh_all/'
):

    copydata = copydata_op(
        data_dir=data_dir,
        checkpoint_dir=checkpoint_dir,
        model_dir='%s/%s/model_output' % (working_dir, dsl.RUN_ID_PLACEHOLDER),
        action=COPY_ACTION,
    )

    train = train_op(
        data_dir=data_dir,
        model_dir=copydata.outputs['copy_output_path'],
        action=TRAIN_ACTION, train_steps=train_steps,
        deploy_webapp=deploy_webapp
    )

    serve = dsl.ContainerOp(
        name='serve',
        image='gcr.io/google-samples/ml-pipeline-kubeflow-tfserve:v6',
        arguments=[
            "--model_name", 'ghsumm-%s' % (dsl.RUN_ID_PLACEHOLDER,),
            "--model_path", train.outputs['train_output_path']
        ]
    )

    train.set_gpu_limit(1)
```

請注意，我們要求「train」步驟在叢集節點上執行，且該節點至少有 1 個可用的 GPU。

```
train.set_gpu_limit(1)
```


管道的最後一個步驟 (同樣是內嵌定義) 是有條件的。只有在訓練步驟 `launch_server` 輸出字串「true」時，這個步驟才會在「`serve`」步驟完成後執行。這會啟動「預測網頁應用程式」，我們曾使用這個應用程式，向訓練後的 T2T 模型要求問題摘要。

```
with dsl.Condition(train.outputs['launch_server'] == 'true'):
    webapp = dsl.ContainerOp(
        name='webapp',
        image='gcr.io/google-samples/ml-pipeline-webapp-launcher:v1',
        arguments=["--model_name", 'ghsumm-%s' % (dsl.RUN_ID_PLACEHOLDER,),
                   "--github_token", github_token]

    )
    webapp.after(serve)
```

元件容器映像檔定義

Kubeflow Pipeline 文件說明瞭建構自有元件的一些[最佳做法](#)。在這個過程中，您需要定義及建構容器映像檔。您可以在[這裡](#)查看本程式碼研究室 pipeline 的元件步驟。Dockerfile 定義位於 `containers` 子目錄中，例如[這裡](#)。

使用搭載 GPU 的先占 VM 進行訓練

先占 VM 是 [Compute Engine VM](#) 執行個體，生命週期最長可達 24 小時，但不提供可用性保證。先占 VM 的價格比標準 Compute Engine VM 低。

使用 [Google Kubernetes Engine \(GKE\)](#)，您可以輕鬆設定[使用可搶占 VM 的叢集或節點集區](#)。您可以設定這類節點集區，並將 GPU 附加至先占執行個體。這些節點的運作方式與一般啟用 GPU 的節點相同，但 GPU 只會在執行個體存留時常駐。

如要為叢集設定啟用 GPU 的先占節點集區，請執行類似下列的指令，並根據您的叢集名稱和區域編輯下列指令，然後根據需求調整加速器類型和數量。您可以選擇定義節點集區，根據目前的工作負載自動調度資源。

```
gcloud container node-pools create preemptible-gpu-pool \
  --cluster=<your-cluster-name> \
  --zone <your-cluster-zone> \
  --enable-autoscaling --max-nodes=4 --min-nodes=0 \
  --machine-type n1-highmem-8 \
  --preemptible \
  --node-taints=preemptible=true:NoSchedule \
  --scopes cloud-platform --verbosity error \
  --accelerator=type=nvidia-tesla-k80,count=4
```

您也可以透過 [Cloud 控制台](#) 設定節點集區。

定義使用可搶占 GKE 節點的 Kubeflow 管道

如果您在 GKE 上執行 Kubeflow，現在可以輕鬆[定義及執行 Kubeflow Pipelines](#)，其中一或多個管道步驟 (元件) [會在可搶占節點上執行](#)，進而降低執行工作的成本。如要使用先佔 VM 取得正確結果，您識別為先佔的步驟應為[冪等](#) (也就是說，如果多次執行步驟，結果會相同)，或應檢查點工作，以便步驟在遭到中斷時從中斷處繼續執行。

定義 Kubeflow Pipeline 時，您可以修改作業，指出特定步驟應在可搶占節點上執行，如下所示：

```
your_pipelines_op.apply(gcp.use_preemptible_nodepool())
```

詳情請參閱[說明文件](#)。

如果節點遭到先占，您可能也會想重試步驟幾次。您可以按照下列方式操作，這裡我們指定 5 次重試。

```
your_pipelines_op.set_gpu_limit(1).apply(gcp.use_preemptible_nodepool()).set_retry(5)
```

嘗試編輯本程式碼研究室中使用的 Kubeflow pipeline，在**先占 VM** 上執行訓練步驟。

在管道規格中變更下列行，額外使用可搶占式節點集區 (請務必如上所示建立節點集區)，並重試 5 次：

```
train.set_gpu_limit(1)
```

接著重新編譯管道、上傳新版本 (並命名)，然後執行新版管道。
